Software Requirements Specification

for

BrowseBlock

AI browser extension for real-time browser based attacks detection and prevention.

Team 16

Prepared by,

Deon George(SJC23CS086)

Amal Kurian Royce (SJC23CS037)

Maria Royce (SJC23CS159)

Melbin K Thomas (SJC23CS164)

St. Joseph's College of Engineering and Technology, Palai

11-10-2026

Table Of Contents

2. Product Scope **1**
2.1 User Classes and Characteristics 2
2.2 Operating Environment 2
2.3 Authentication and Application Data Management 3

3. Objectives **4**

4. System Features **4**
4.1 URL and Fake QR Phishing 4
4.2 Cross-Site Scripting (XSS) 5
4.3 Credential Harvesting 7
4.4 HTML/JavaScript Redirects 7
4.5 Clickjacking 8
4.6 Cryptojacking 9
4.7 Malicious File Download Detection 10
4.8 Email Phishing Detection 12
4.9 Browser Notification Scams 13
4.10 Browser Extension Abuse 14

1 URL / Fake QR Phishing **15**

2 HTML / JavaScript Redirects **15**

3 Malicious File Downloads **16**

4 E-mail Phishing **17**

5 Cryptojacking **18**

6 XSS scripting **19**

7 Clickjacking **20**

8 Browser Extension Abuse **21**

9 Browser Notification Scams **23**

10 Credential Harvesting **24**

2. Product Scope

BrowseBlock is an AI/ML-powered browser security platform(both browser extension and web dashboard) designed to detect, analyze, explain, and help prevent browser-based cyber threats in real time. The system consists of a browser extension, a FastAPI backend for centralized security processing, and a React-based web dashboard for security monitoring and reporting. Firebase Authentication manages user registration, sign-in, and identity verification, while MongoDB stores application data such as user-related records, scan history, detection results, threat reports, and system records. The FastAPI backend coordinates security processing, verifies authenticated requests, and accesses MongoDB according to the user's authorization.

The browser extension acts as the primary protection layer by monitoring supported browsing activities, evaluating URLs, inspecting webpage content and behaviour, identifying suspicious redirects, and monitoring download events. It provides protection against ten threat categories: URL and fake QR phishing, Cross-Site Scripting (XSS), credential harvesting, HTML/JavaScript redirects, clickjacking, cryptojacking, malicious file downloads, email phishing, browser notification scams, and browser extension abuse.

The URL phishing detection module uses XGBoost to classify URLs based on extracted URL and domain features. For QR-based phishing detection, the system decodes supported QR-code inputs and evaluates the embedded URLs using the URL-analysis module.

The email phishing detection module uses Random Forest-based screening and available email-authentication signals within the browser extension for initial threat assessment. For detailed analysis through the web dashboard, a Retrieval-Augmented Generation (RAG) pipeline analyzes the email content, extracts URLs, and identifies suspicious patterns. The extracted URLs are further evaluated using BrowseBlock's URL phishing detection module. This two-stage approach enables lightweight, real-time screening in the browser extension while providing deeper semantic analysis and comprehensive risk assessment through the web dashboard when required.

Other protection modules analyze webpage scripts and behaviour, suspicious form submissions, redirect chains, framing patterns, notification-related scam indicators, and observable browser extension activity. Potentially malicious downloads in supported formats, including .zip, .pdf, .exe, .doc, and .docx, are assessed using file-type-specific static analysis, machine-learning models, and security rules. Untrusted files are not executed simply for analysis.

The browser extension remains lightweight and event-driven, while computationally intensive analysis is handled by the FastAPI backend where appropriate. The extension provides warnings or blocks risky activities when supported by browser APIs, permissions, and available evidence. These mechanisms provide an additional security layer but cannot guarantee prevention of every browser-based attack.

The React web dashboard enables users to manually scan URLs, QR codes, emails, and supported files; review detection history; examine threat reports and analytics; and understand the reasons, indicators, and evidence behind security decisions. Scan results, detection history, threat reports, and applicable system records are stored in MongoDB so that authorized users can retrieve relevant information beyond the current session. Access to stored information shall be restricted according to the authenticated user's identity and

permissions.

The primary objective of BrowseBlock is to provide a unified, explainable, and user-friendly browser security solution for individuals and organizations. It aims to reduce exposure to phishing attacks and malicious web activities without replacing existing antivirus software, browser security features, or enterprise security solutions.

2.1 User Classes and Characteristics

Primary Users: General internet users, students, employees, and individuals who browse websites, access web-based email services, encounter QR codes, and download files. These users may have limited cybersecurity knowledge and require clear warnings, understandable explanations, and practical security recommendations.

Organizational Users: Small businesses, educational institutions, and other organizations seeking an additional layer of protection against phishing emails, fraudulent websites, malicious QR codes, suspicious webpage behaviour, and unsafe downloads.

Security Analysts: Authorized users who review suspicious activities, examine detection evidence, interpret risk assessments, review threat reports, and investigate security incidents through the web dashboard.

System Administrators: Personnel responsible for managing system configurations, maintaining backend services, monitoring system performance, managing authentication-related configuration, and ensuring secure, authorized access to stored security records.

The system is intended to provide an accessible interface for non-technical users while offering detailed threat information and detection evidence for users who require more comprehensive security analysis.

2.2 Operating Environment

Browser Environment: Google Chrome and compatible Chromium-based browsers that support the required browser-extension APIs. Manifest V3 is the intended extension architecture, subject to the availability and limitations of the required APIs.

Web Dashboard: A responsive React-based web application accessible through modern desktop web browsers.

Frontend Technologies: React and Tailwind CSS for the dashboard interface.

Backend Environment: Python with FastAPI for REST API development, request validation, scan-request processing, coordination of detection modules, and communication with machine-learning and AI services. FastAPI endpoints expose the required backend functionality to the browser extension and web dashboard. The backend shall verify Firebase Authentication ID tokens for protected requests, enforce authorization, and interact with MongoDB for authorized data operations. The Firebase Admin SDK may be used for server-side token verification. Asynchronous request handling may be used where appropriate for I/O-bound operations.

Machine-Learning Environment: Python-based machine-learning components, including XGBoost for URL phishing detection and Random Forest for initial email phishing screening. Type-specific static analysis, machine-learning models, and deterministic security rules support malicious file assessment. SHAP is used where appropriate to explain relevant tree-model predictions.

AI-Based Email Analysis: RAG and LLM techniques support deeper semantic anal-

ysis of suspicious emails, including social engineering, impersonation, urgency, credential harvesting, and phishing intent. Available email-authentication signals are incorporated into the overall risk assessment where applicable.

Database Environment: MongoDB is used as the application's database for storing relevant user profile metadata, scan history, detection results, threat reports, and system records. Firebase Authentication manages user authentication and credentials; passwords and authentication secrets shall not be stored in MongoDB. Application data shall be associated with the appropriate authenticated user or authorized role, and database access shall be restricted accordingly.

Network Requirements: Network connectivity is required for communication between the browser extension, the FastAPI backend, the web dashboard, Firebase Authentication, MongoDB, and any enabled external threat-intelligence or AI services. Access to external services depends on their configuration and availability. MongoDB access shall be restricted to authorized backend services rather than exposed directly to untrusted clients.

Security Requirements: The system should use HTTPS for network communication, Firebase Authentication for user sign-in and identity verification, server-side verification of Firebase ID tokens by FastAPI, and appropriate authorization checks for protected endpoints and stored records. MongoDB access must be restricted, and queries returning user-specific data must enforce ownership or role-based access controls. URLs, email content, downloaded files, authentication tokens, and other potentially sensitive information must be handled securely. Data collection and retention should be minimized to protect user privacy.

2.3 Authentication and Application Data Management

User Authentication: Firebase Authentication shall be used for user registration, sign-in, and identity verification. The frontend shall use the Firebase Authentication client integration appropriate to the application. Protected requests to FastAPI shall include the Firebase ID token, and the backend shall validate the token server-side before processing the request.

Application Data: MongoDB shall store applicable user profile metadata, scan history, detection results, threat reports, and system records. Authentication credentials and passwords shall remain managed by Firebase Authentication and shall not be stored in MongoDB.

Authorization and Privacy: Data access shall be restricted to the authenticated user or other explicitly authorized roles. The backend shall enforce authorization before reading or changing user-specific MongoDB records. Sensitive URL parameters, email content, webpage contents, downloaded file contents, and authentication tokens shall not be retained unnecessarily. The retention of scan records and reports shall follow the system's configured privacy and retention policy.

3. Objectives

Real-Time Threat Handling: Detect, analyze, explain, and prevent major browser-based cyber threats in real time.

Active User Protection: Provide active protection while the user browses using a browser extension by monitoring events, issuing warnings, and blocking harmful actions whenever possible.

Core Security Philosophy Execution: Implement a security lifecycle focused on detecting threats, assessing risk, explaining why they are dangerous, and preventing harmful actions.

Centralized Analysis and Management: Support client-side protection with a FastAPI backend for centralized security processing, deep model inference, model management, and authorized interaction with MongoDB.

Visibility and Reporting: Provide centralized visibility into overall security posture, threat statistics, analytics, incident details, persistent detection history, and manual scanning capabilities through the web dashboard, using records stored in MongoDB and enforcing user-specific access controls.

Proportional Risk Evaluation: Move away from simple binary detection by utilizing a tiered risk decision model (Safe/allow, Suspicious/warn, High-risk/block) based on evaluated threat risk levels.

Secure Authentication and Access Control: Use Firebase Authentication for user registration, sign-in, and identity verification. FastAPI shall validate Firebase ID tokens and enforce authorization before allowing access to protected endpoints or user-specific MongoDB records.

Balancing Performance and Security: Maintain a lightweight, event-driven client layer that safeguards user privacy and separates AI/ML risk assessment from deterministic browser enforcement.

4. System Features

4.1 URL and Fake QR Phishing

4.1.1 Description

Evaluates URLs using an XGBoost classifier across normalized numerical features (such as URL length, IP usage, domain age, subdomains, and brand impersonation indicators), and decodes QR codes to evaluate the underlying URL payload to warn or block navigation.

4.1.2 Functional Requirements

REQ-01 (URL Monitoring): The system shall monitor supported browser navigation events and identify URLs requiring phishing analysis.

REQ-02 (URL Normalization): The system shall normalize URLs before extracting features for phishing classification.

REQ-03 (Feature Extraction): The system shall extract configured URL and domain features, including URL length, subdomains, IP-address usage, special characters, suspicious keywords, and available domain information.

REQ-04 (XGBoost Classification): The system shall use an XGBoost classifier to assess the phishing risk of supported URLs.

REQ-05 (QR Code Decoding): The system shall decode QR codes submitted through supported scanning workflows and extract their payloads.

4.1.3 Non-Functional Requirements

REQ-01 (Processing Efficiency): The system shall perform URL preprocessing and classification efficiently to minimize unnecessary browsing delays.

REQ-02 (Detection Accuracy): The URL detector shall be evaluated using appropriate validation data to measure precision, recall, and false-positive rates.

REQ-03 (Input Robustness): The system shall handle malformed, unusually long, and encoded URLs without crashing.

REQ-04 (Privacy): The system shall minimize the storage of sensitive URL parameters and unnecessary browsing information.

4.2 Cross-Site Scripting (XSS)

4.2.1 Description

Detects potential Cross-Site Scripting (XSS) attacks by inspecting webpage content, JavaScript behavior, and DOM manipulation for suspicious script injection indicators. The system addresses three types of XSS attacks: reflected XSS, stored XSS, and DOM-based XSS. Detection techniques include identifying suspicious URL parameters, injected script patterns, dynamically generated code, obfuscated JavaScript, unsafe DOM sinks, and unexpected script sources. BrowseBlock uses configured detection rules and observable browser behavior to assess potential threats, warn users, and restrict suspicious interactions where browser capabilities permit.

4.2.2 Functional Requirements

REQ-10 (Webpage Inspection): The system shall inspect supported webpage content, URL parameters, and observable JavaScript behavior for indicators associated with XSS attacks.

REQ-11 (Script Pattern Detection): The system shall identify configured suspicious patterns involving inline scripts, dynamically generated code, obfuscated JavaScript, and unexpected script sources.

REQ-12 (DOM Behavior Analysis): The system shall analyze observable DOM manipulation patterns and potentially unsafe source-to-sink flows associated with client-side script injection.

REQ-13 (Reflected XSS Detection): The system shall identify potential reflected XSS by examining URL parameters and other request-derived inputs for suspicious script injection patterns and their observable reflection in webpage content.

REQ-14 (Stored XSS Detection): The system shall identify observable indicators of potential stored XSS, including suspicious injected content rendered in webpages and

persistent script patterns where inspection capabilities permit. Detection shall account for content that may originate from stored user submissions or other server-provided data.

REQ-15 (DOM-Based XSS Detection): The system shall identify potential DOM-based XSS by inspecting observable client-side behavior involving untrusted inputs and potentially unsafe DOM or JavaScript execution sinks.

REQ-16 (XSS Risk Assessment): The system shall evaluate detected indicators using configured rules and available analysis components to determine the potential severity of an XSS threat.

REQ-17 (Reflected XSS Prevention): The system shall warn users about suspected reflected XSS and block known malicious URLs or restrict navigation where supported browser capabilities and reliable detection rules permit.

REQ-18 (Stored XSS Prevention): The system shall warn users about suspected stored XSS content and restrict access to affected pages or known malicious resources where supported browser capabilities permit. The system shall not assume that a browser extension can remove the underlying stored payload from the server.

REQ-19 (DOM-Based XSS Prevention): The system shall warn users about suspected DOM-based XSS and restrict identified malicious navigation or resource loading where supported. The system shall apply available browser-level blocking mechanisms without relying solely on removing DOM elements after potentially harmful scripts have executed.

REQ-20 (XSS Warning): The system shall provide users with a warning when supported inspection identifies potentially dangerous script injection or execution behavior.

REQ-21 (Interaction Protection): The system shall restrict suspicious client-side interactions using supported browser extension capabilities and configured protection rules.

REQ-22 (Detection Logging): The system shall record relevant detection indicators, the identified XSS category where determinable, the risk assessment, and the protective action without unnecessarily storing complete webpage contents.

4.2.3 Non-Functional Requirements

REQ-05 (Performance): The system shall inspect webpage content and observable JavaScript behavior without introducing unnecessary page-loading or interaction delays.

REQ-06 (Detection Reliability): The detector shall minimize false positives involving legitimate JavaScript, URL parameters, dynamic content, and normal DOM manipulation.

REQ-07 (Privacy): The system shall minimize the collection and storage of webpage content, URL parameters, and other potentially sensitive data during XSS detection.

REQ-08 (Detection Coverage): The system shall implement distinct detection rules and test cases for reflected XSS, stored XSS, and DOM-based XSS, subject to browser visibility and extension permission limitations.

REQ-09 (Prevention Limitations): The system shall distinguish between detecting suspicious behavior and preventing script execution, and shall not report an attack as blocked unless the configured protective action has actually been applied.

4.3 Credential Harvesting

4.3.1 Description

Detects potential credential-harvesting attempts by analyzing suspicious external script loading, deceptive login interfaces, and scam pop-ups. Playwright is used in the analysis and testing workflow to automate webpage interactions, monitor network requests, identify external script domains, and investigate suspicious pop-up behavior. The collected indicators support risk assessment and the development of blocking rules within the BrowseBlock browser extension.

4.3.2 Functional Requirements

REQ-17 (External Script Monitoring): The system shall identify and monitor external domains loading JavaScript resources on webpages using Playwright-based analysis.

REQ-18 (Suspicious Domain Detection): The system shall evaluate external script domains against configured indicators, domain reputation information, and available threat intelligence to identify potentially malicious resources.

REQ-19 (Scam Pop-up Detection): The system shall identify suspicious pop-ups and dynamically generated browser-page overlays that exhibit configured scam indicators.

REQ-20 (Credential-Harvesting Indicators): The system shall analyze observable webpage behavior, including suspicious scripts, deceptive login interfaces, and pop-ups requesting sensitive information, to assess potential credential-harvesting risk.

REQ-21 (Risk Assessment): The system shall combine relevant script-domain, webpage, and pop-up indicators to determine the potential risk of credential harvesting.

REQ-22 (Scam Pop-up Blocking): The browser extension shall warn users about or block detected scam pop-ups using supported browser capabilities and configured detection rules.

REQ-23 (Suspicious Resource Blocking): The system shall block identified malicious script URLs or domains where reliable blocking rules and browser permissions are available.

REQ-24 (Detection Logging): The system shall record detected indicators, risk assessments, and protective actions without collecting or logging actual passwords.

4.3.3 Non-Functional Requirements

REQ-05 (Response Time): The system shall minimize delays during script-domain inspection and scam pop-up detection.

REQ-06 (Credential Privacy): The system shall not collect or log actual passwords during its normal detection process.

REQ-10 (Detection Accuracy): The detector shall minimize false positives involving legitimate third-party scripts, authentication providers, and normal pop-up behavior.

4.4 HTML/JavaScript Redirects

4.4.1 Description

Monitors HTTP redirects, JavaScript-based navigation, and meta-refresh tags to trace redirect chains, evaluate destination reputation, and interrupt malicious navigation.

4.4.2 Functional Requirements

REQ-25 (HTTP Redirect Monitoring): The system shall monitor supported HTTP redirects and browser navigation events.

REQ-26 (Client-Side Redirect Detection): The system shall inspect observable JavaScript navigation and HTML meta-refresh indicators.

REQ-27 (Redirect Chain Tracking): The system shall maintain an observable sequence of source and destination URLs associated with a redirect chain.

REQ-28 (Destination Reputation Analysis): The system shall evaluate redirect destinations using the URL phishing detector and available threat-intelligence information.

REQ-29 (Redirect Risk Assessment): The system shall identify redirect chains that contain configured indicators of potentially malicious navigation.

REQ-30 (Malicious Navigation Prevention): The system shall warn about or interrupt navigation to high-risk destinations where supported.

REQ-31 (Redirect Event Logging): The system shall record relevant redirect information, the final risk assessment, and the action taken.

4.4.3 Non-Functional Requirements

REQ-11 (Navigation Performance): The system shall analyze redirect chains efficiently to minimize unnecessary navigation delays.

REQ-12 (Monitoring Reliability): The system shall handle incomplete or unavailable redirect information without falsely reporting that the entire redirect chain was analyzed.

REQ-13 (Resource Efficiency): The system shall avoid unnecessary repeated analysis of the same redirect event.

4.5 Clickjacking

4.5.1 Description

Identifies deceptive framing behavior and suspicious iframe setups to block risky interactions or issue warnings, complementing server-side defenses like CSP frame-ancestors and X-Frame-Options.

4.5.2 Functional Requirements

REQ-32 (Iframe Inspection): The system shall inspect observable iframe attributes and webpage framing structures.

REQ-33 (Suspicious Frame Detection): The system shall identify configured indicators of deceptive framing, suspicious positioning, and potentially misleading interactive elements.

REQ-34 (Interaction Risk Assessment): The system shall evaluate detected indicators to determine potential clickjacking risk.

REQ-35 (Clickjacking Warning): The system shall warn users when supported inspection identifies suspicious framing behavior.

REQ-36 (Interaction Protection): The system shall block or restrict risky interactions where a reliable prevention mechanism is available through supported browser

capabilities.

REQ-37 (Detection Logging): The system shall record relevant indicators, risk assessments, and protective actions.

4.5.3 Non-Functional Requirements

REQ-14 (Interaction Performance): The system shall perform frame inspection without significantly affecting normal webpage interactions.

REQ-15 (Detection Reliability): The detector shall minimize false positives involving legitimate iframe usage and embedded content.

REQ-16 (Browser Compatibility): The system shall implement clickjacking protections within the capabilities and permissions of the selected browser.

4.6 Cryptojacking

4.6.1 Description

Inspects scripts, WebAssembly behaviors, web worker activities, and abnormal resource-usage patterns to identify and block unauthorized cryptocurrency mining in the browser.

4.6.2 Functional Requirements

REQ-38 (Mining Script Detection): The system shall inspect observable script indicators associated with unauthorized cryptocurrency mining.

REQ-39 (WebAssembly and Worker Analysis): The system shall analyze supported WebAssembly and web worker indicators for suspicious mining-related behavior.

REQ-40 (Mining Resource Identification): The system shall identify configured indicators associated with known cryptocurrency-mining resources.

REQ-41 (Resource Usage Analysis): The system shall evaluate available browser resource-usage indicators when supported.

REQ-42 (Cryptojacking Risk Assessment): The system shall combine relevant indicators to determine a cryptojacking risk category.

REQ-43 (Cryptojacking Protection): The system shall warn users about detected risks or block suspicious resources where browser capabilities permit.

REQ-44 (Activity Logging): The system shall record relevant detection evidence and the action taken.

4.6.3 Non-Functional Requirements

REQ-17 (Resource Efficiency): The system shall minimize additional CPU and memory consumption during cryptojacking detection.

REQ-18 (Detection Accuracy): The detector shall minimize false positives involving legitimate computationally intensive webpages.

REQ-19 (Monitoring Limitations): The system shall restrict its conclusions to script behavior and resource information observable through supported browser mechanisms.

4.7 Malicious File Download Detection

4.7.1 Description

Implements a malicious file download detection pipeline that monitors supported downloads of `.exe`, `.zip`, `.pdf`, `.doc`, and `.docx` files through the browser extension. The system uses content-based file identification, format-specific static analysis, SHA-256 hash calculation, available threat-intelligence checks, deterministic rules, and XGBoost classification to assess file risk without executing untrusted files. The FastAPI backend coordinates analysis and risk assessment, while a native helper application can provide access to file contents and support controlled quarantine and release workflows. Based on the assessment, BrowseBlock shall allow, warn about, cancel, or quarantine files where supported by the selected integration.

4.7.2 Functional Requirements

REQ-45 (Download Monitoring): The browser extension shall monitor supported download events and collect available metadata, including the download URL, filename, MIME type, and download identifier.

REQ-46 (File Type Identification): The system shall identify supported `.exe`, `.zip`, `.pdf`, `.doc`, and `.docx` files using filename extensions, MIME types, and content-based identification where file access is available.

REQ-47 (Executable Analysis): The system shall extract configured executable features, including PE headers, section characteristics, entropy, imported APIs, digital signature information, and resources where applicable.

REQ-48 (PDF Analysis): The system shall inspect PDF structures for potentially suspicious JavaScript, `OpenAction` entries, embedded files, objects, streams, and external URLs.

REQ-49 (Document Analysis): The system shall inspect supported DOC and DOCX files for applicable macros, OLE objects, embedded files, external relationships, and suspicious document structures.

REQ-50 (Archive Analysis): The system shall inspect ZIP archive metadata, contained file types, suspicious paths, compression characteristics, and nested archives within configured analysis limits.

REQ-51 (Content-Based File Verification): The system shall verify file types using available content-based identification and shall not rely solely on filename extensions or MIME types.

REQ-52 (Feature Extraction): The system shall extract configured numerical and categorical features from supported file formats and produce structured records suitable for classification.

REQ-53 (Hash and Reputation Analysis): The system shall calculate a SHA-256 hash for accessible files and check available threat-intelligence or file-reputation sources where configured.

REQ-54 (File Risk Classification): The system shall evaluate extracted features using a configured XGBoost classifier, deterministic detection rules, and available hash-reputation results.

REQ-55 (Risk Decision): The system shall classify analyzed files as low-risk, suspicious, or high-risk according to the configured decision policy, taking into account analysis

completeness and available evidence.

REQ-56 (Download Protection): The browser extension shall warn about or cancel supported downloads according to the risk assessment and available browser capabilities. Where pre-use protection is required, the system shall support a controlled workflow through a native helper application or download gateway.

REQ-57 (File Quarantine and Release): Where a native helper or controlled gateway is deployed, the system shall support holding suspicious files in a controlled location until analysis is complete and shall permit release or quarantine according to the configured risk policy.

REQ-58 (Analysis Coordination): The system shall use the FastAPI backend to coordinate file-analysis requests, feature-processing workflows, classification, and the return of assessment results where backend processing is required.

REQ-59 (Analysis Failure Handling): The system shall identify unsupported formats, malformed files, parser errors, timeouts, and incomplete analysis, and shall apply the configured warning or hold policy rather than automatically treating failed analysis as a safe result.

REQ-60 (File Analysis Logging): The system shall record the file type, timestamp, available hash, risk assessment, relevant detection indicators, and protective action, subject to configured privacy and retention policies.

4.7.3 Non-Functional Requirements

REQ-20 (Safe File Processing): The system shall analyze untrusted files without executing them and shall isolate file parsers and analysis components where practical.

REQ-21 (Resource Limits): The system shall enforce configurable limits on file size, processing time, archive depth, decompressed size, entry count, and resource consumption.

REQ-22 (Processing Reliability): The system shall handle malformed files, unsupported formats, parser failures, and analysis timeouts without crashing the extension, backend, or analysis service.

REQ-23 (Privacy and Retention): The system shall minimize the transmission and retention of downloaded file contents, retaining only the information required for detection, auditing, and configured security purposes.

REQ-24 (Type Verification): The system shall use content-based file identification where file bytes are available and shall treat discrepancies between the filename extension, MIME type, and actual file format as potential risk indicators rather than definitive proof of maliciousness.

REQ-30 (Analysis Isolation): The system shall isolate file-processing components from the main application where practical and shall prevent untrusted files from being executed during static analysis.

REQ-31 (Protection Reliability): The system shall distinguish between download monitoring, file analysis, and enforced blocking or quarantine, and shall report a protective action as successful only when the action has actually been confirmed.

REQ-32 (Model Reliability): The system shall evaluate the XGBoost classifier using appropriate validation data and shall not rely on model predictions alone when making high-impact file-blocking decisions.

REQ-33 (Integration Compatibility): The system shall implement download monitoring and enforcement according to the capabilities and permissions of the selected

browser APIs, native helper, or controlled download gateway.

4.8 Email Phishing Detection

4.8.1 Description

Implements a two-stage email phishing detection pipeline. The browser extension performs lightweight initial screening using a Random Forest classifier and available email-authentication signals, including SPF, DKIM, and DMARC results. For detailed investigation, the web dashboard uses a Retrieval-Augmented Generation (RAG) pipeline to analyze email content, identify social engineering tactics, and extract URLs. The extracted URLs are evaluated using BrowseBlock's URL phishing detection module. This architecture enables efficient initial screening in the browser extension while providing deeper contextual analysis through the web dashboard when required.

4.8.2 Functional Requirements

REQ-56 (Email Input): The system shall accept email content through supported browser extension integrations or manual submission workflows in the web dashboard.

REQ-57 (Email Feature Extraction): The system shall extract configured email features, including message length, subject characteristics, URL count, attachment presence, suspicious keyword counts, and sender-domain mismatch, where the required data is available.

REQ-58 (Random Forest Screening): The browser extension shall use a Random Forest classifier to perform lightweight initial email phishing assessment using configured email features.

REQ-59 (Email Authentication Signals): The browser extension shall incorporate available SPF, DKIM, and DMARC authentication results into the initial email risk assessment.

REQ-60 (Initial Risk Assessment): The browser extension shall combine the Random Forest prediction and available email-authentication signals to identify potentially suspicious emails and provide an initial risk assessment.

REQ-61 (Email Submission for Detailed Analysis): The system shall allow users to submit email content to the web dashboard for detailed phishing analysis when required.

REQ-62 (RAG-Based Email Analysis): The web dashboard shall use a Retrieval-Augmented Generation (RAG) pipeline to retrieve relevant information from a configured phishing knowledge base and support the contextual analysis of submitted email content.

REQ-63 (Email URL Extraction): The RAG-based analysis workflow shall identify and extract supported URLs from submitted email content for further security assessment.

REQ-64 (URL Phishing Analysis): The system shall evaluate extracted URLs using BrowseBlock's URL phishing detection module and incorporate the resulting risk information into the detailed email assessment.

REQ-65 (Semantic Threat Assessment): The configured LLM component within the RAG workflow shall analyze email content for indicators of social engineering, impersonation, urgency, credential harvesting, and malicious intent.

REQ-66 (Detailed Risk Classification): The web dashboard shall combine available initial screening results, URL analysis results, and contextual analysis evidence to

classify the email as safe, suspicious, or high-risk according to the configured decision policy.

REQ-67 (Email Warning): The system shall warn users about suspicious or high-risk emails and present available supporting evidence.

REQ-68 (Email Link Protection): The system shall warn users about or restrict access to associated email links and actions where supported by the browser integration.

REQ-69 (Email Event Logging): The system shall record the assessment result, timestamp, available evidence, and protective action, subject to configured privacy and retention policies.

4.8.3 Non-Functional Requirements

REQ-25 (Analysis Efficiency): The system shall perform lightweight initial screening within the browser extension and reserve resource-intensive RAG and LLM analysis for detailed investigations through the web dashboard.

REQ-26 (Processing Timeouts): The system shall apply configurable timeouts to RAG retrieval, LLM inference, and external service requests.

REQ-27 (Email Privacy): The system shall minimize the transmission and retention of sensitive email content and obtain user authorization where required before submitting email content for detailed analysis.

REQ-28 (Authentication Signal Reliability): The system shall not treat successful SPF, DKIM, or DMARC checks as definitive proof that an email is safe.

REQ-29 (Failure Handling): The system shall distinguish incomplete or failed email analysis from a completed safe verdict and shall communicate analysis limitations where appropriate.

4.9 Browser Notification Scams

4.9.1 Description

Detects deceptive notification requests and prompts imitating urgent system alerts, prizes, or fake security warnings, issuing warnings or blocking interactions when browser APIs allow.

4.9.2 Functional Requirements

REQ-68 (Notification Request Monitoring): The system shall monitor supported webpage notification-permission requests and related prompts.

REQ-69 (Deceptive Prompt Detection): The system shall identify configured indicators of fake security warnings, prize scams, urgent alerts, and deceptive notification requests.

REQ-70 (Notification URL Analysis): The system shall evaluate relevant URLs associated with detected notification-related scams.

REQ-71 (Notification Scam Assessment): The system shall assess observable notification-related indicators and determine a risk category.

REQ-72 (Notification Protection): The system shall warn users about suspicious notification-related interactions and restrict associated actions where browser capabilities permit.

REQ-73 (Notification Event Logging): The system shall record relevant detection evidence and protective actions.

4.9.3 Non-Functional Requirements

REQ-30 (Usability): The system shall minimize interference with legitimate website notification functionality.

REQ-31 (Detection Reliability): The system shall minimize false positives involving legitimate notification prompts.

REQ-32 (Monitoring Scope): The system shall limit its conclusions to notification-related content and behavior observable through supported browser mechanisms.

4.10 Browser Extension Abuse

4.10.1 Description

Identifies suspicious browser extensions by monitoring overly broad permissions, unexpected webpage tampering, and malicious script injection through observable browser security mechanisms.

4.10.2 Functional Requirements

REQ-74 (Extension Behavior Monitoring): The system shall monitor observable browser and webpage indicators associated with potential extension abuse.

REQ-75 (Permission Analysis): The system shall evaluate extension-permission information where the browser exposes the required information through supported APIs.

REQ-76 (Webpage Tampering Detection): The system shall identify configured indicators of unexpected webpage manipulation and suspicious script injection.

REQ-77 (Abuse Risk Assessment): The system shall classify observed indicators using configured detection rules and available evidence.

REQ-78 (Abuse Warning): The system shall warn users about potentially abusive extension behavior when sufficient observable evidence is available.

REQ-79 (Abuse Event Logging): The system shall record available indicators and the resulting risk assessment.

4.10.3 Non-Functional Requirements

REQ-33 (Monitoring Efficiency): The system shall minimize additional browser resource consumption during extension-abuse detection.

REQ-34 (Detection Scope): The system shall limit its conclusions to extension behavior and permission information observable through supported browser mechanisms.

REQ-35 (Privacy): The system shall minimize the collection of unrelated browsing information during extension-abuse detection.

Literature Review

1 URL / Fake QR Phishing

a) Summary of the Paper: "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation" by Milosz Misiek and Tomasz Hyla, proposes a machine learning approach for identifying phishing URLs using the XGBoost algorithm. The method extracts URL features and classifies web addresses as legitimate or malicious. The model is evaluated on three different datasets to assess its accuracy and ability to generalize to unseen data. It achieves 90.7% accuracy and a 91.2% F1-score on the custom dataset, with an average accuracy of 77.84% across three test datasets. The study highlights XGBoost's computational efficiency, its ability to operate without GPU acceleration, and its feature-importance analysis. However, detection performance decreases across different datasets, emphasizing the need for reliable feature selection, continuous model updates, and further validation.

b) Relevance to BrowseBlock: This paper directly supports BrowseBlock's URL / Fake QR Phishing Detection feature by providing an XGBoost-based approach for identifying malicious URLs using extracted URL characteristics. BrowseBlock can adopt this approach by normalizing URLs, extracting relevant features, and using the trained XGBoost model to estimate phishing risk. URLs decoded from QR codes can undergo the same analysis, allowing BrowseBlock to warn users about or block suspicious destinations. The model's computational efficiency makes it suitable for integration with BrowseBlock's browser extension and backend architecture. However, QR-code decoding must be implemented separately, and the model should be tested on diverse, up-to-date datasets to evaluate its generalization and minimize false positives before deployment.

2 HTML / JavaScript Redirects

a) Summary of the Paper: The paper "Hidden Path: Understanding the Intermediary in Malicious Redirections" (2022) by Yuwei Zeng, Zhicheng Liu, Xunxun Chen, and Tianning Zang investigates how attackers use URL redirections to hide malicious websites and bypass security checks. The authors develop a detection framework that identifies suspicious intermediary domains, uses a dynamic Chrome-based crawler to observe webpage behavior, and reconstructs redirection chains from browser performance

logs. The study examines HTTP-based, JavaScript-based, and HTML-based redirects, including delayed redirects, fabricated user clicks, and iframe-based techniques. The findings show that malicious redirections often involve multiple hops and diverse domains, with JavaScript playing a significant role. The paper provides practical methods for identifying concealed redirection paths and understanding how users are redirected to malicious destinations.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's HTML / JavaScript Redirects feature because it explains how attackers use intermediate URLs, multiple redirections, delayed navigation, and JavaScript-based techniques to conceal malicious destinations. BrowseBlock can adopt the paper's redirection-chain monitoring and reconstruction approach to track navigation paths and identify suspicious HTTP, JavaScript, and HTML redirects. Additional methodologies can include Chrome's webNavigation API for monitoring real-time navigation events, URL reputation checking, redirect risk scoring, and declarativeNetRequest rules for blocking malicious destinations. These techniques can help BrowseBlock classify navigation as safe, suspicious, or malicious and warn users or prevent risky navigation where browser capabilities permit. However, the paper's large-scale crawling framework cannot be directly transferred to a lightweight browser extension and must be adapted for real-time operation.

3 Malicious File Downloads

a) Summary of the Paper: The paper *"Hidost: A Static Machine-Learning-Based Detector of Malicious Files"* (2016) by Nedim Šrndić and Pavel Laskov presents a static malware detection system that identifies malicious files by analyzing their logical structure and content without executing them. The authors develop Hidost, which extracts structural features from files, consolidates similar structural paths, selects relevant features, and applies machine learning to classify files as benign or malicious. The system is evaluated on PDF and SWF (Flash) files using datasets containing approximately 440,000 PDF files and 40,000 SWF files. The results demonstrate strong detection performance, with Hidost detecting more malicious PDF files than the antivirus engines evaluated through VirusTotal and ranking among the best for SWF malware. The study highlights the potential of structural feature extraction and static machine learning for efficient malware detection while discussing limitations such as dynamically loaded threats

and the need to update detection models as malware patterns evolve.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Malicious File Download Detection** feature because it demonstrates how static analysis and machine learning can identify malicious content within downloaded files without executing them. BrowseBlock can apply similar principles by extracting file-specific features, such as PDF JavaScript actions, embedded files, document structures, executable file properties, and suspicious archive contents, before classifying files using its proposed XGBoost model and deterministic security rules. The paper's structural path consolidation technique also provides a useful research direction for reducing feature complexity and improving the representation of structurally similar files. These approaches can help BrowseBlock assess downloaded files and allow, warn about, or block potentially malicious content where browser capabilities permit. However, Hidost was evaluated on PDF and SWF formats, so its results cannot be directly generalized to BrowseBlock's other target formats, including EXE, ZIP, DOC, and DOCX. Each format requires suitable feature extraction, training data, and independent evaluation.

4 E-mail Phishing

a) Summary of the Paper: The paper *"Classification of Phishing Email Using Random Forest Machine Learning Technique"* (2014) by Andronicus A. Akinyelu and Aderemi O. Adewumi investigates the use of the Random Forest machine-learning algorithm to classify emails as phishing or legitimate. The authors extract 15 phishing-related features from email messages, including IP-based URLs, mismatches between link text and actual destinations, suspicious keywords, JavaScript presence, the number of links, and differences between the sender's domain and domains referenced in the email. The extracted features are represented numerically and used to train and evaluate a Random Forest classifier through 10-fold cross-validation. Experiments conducted using a dataset of 2,000 emails report a classification accuracy of 99.7%, a false-positive rate of 0.06%, and a false-negative rate of 2.50% for the largest dataset configuration. The study demonstrates the potential of feature-based machine learning for phishing email detection while emphasizing the importance of selecting relevant phishing-specific features to improve classification performance.

b) Relevance to BrowseBlock: This paper is directly relevant to BrowseBlock's **Email Phishing Detection** feature because it demonstrates how a Random Forest classifier can identify phishing emails using measurable characteristics of email content and embedded links. BrowseBlock can adopt similar features, including suspicious keywords, URL counts, IP-based URLs, mismatches between displayed link text and actual destinations, and sender-domain inconsistencies, to build its initial email-screening model. The resulting Random Forest prediction can be combined with BrowseBlock's URL phishing detection model, email authentication signals such as SPF, DKIM and DMARC, and RAG/LLM-based semantic analysis to produce a more comprehensive risk assessment. Based on the final verdict, BrowseBlock can display a warning, flag suspicious messages, and block malicious links or associated navigation where browser capabilities permit. However, the paper was published in 2014 and evaluated a relatively small dataset, so its reported accuracy should not be assumed to represent performance on modern phishing attacks. BrowseBlock must train and evaluate its model on representative, up-to-date datasets and measure false positives and false negatives before deployment.

5 Cryptojacking

a) Summary of the Paper: The paper *"Analyzing In-Browser Cryptojacking"* (2024) by Muhammad Saad and David Mohaisen investigates cryptojacking attacks in which malicious JavaScript secretly uses website visitors' computing resources to mine cryptocurrencies without their consent. The authors analyze more than 5,700 websites associated with cryptojacking and examine the characteristics of malicious mining scripts through static and dynamic analysis. For static analysis, they extract JavaScript code features, including cyclomatic complexity, Halstead complexity measures, and source-code characteristics, and apply machine-learning algorithms such as Logistic Regression, Linear Discriminant Analysis, Support Vector Machines, k-Nearest Neighbors, and Random Forest to distinguish cryptojacking scripts from benign and other malicious JavaScript. Their experiments report strong classification performance, with several models achieving 100% precision, recall, and F1-score on the evaluated datasets. Dynamic analysis investigates the impact of cryptojacking on CPU consumption, battery drainage, and WebSocket communication with remote mining servers. The findings show that cryptojacking can significantly consume system resources and drain device batteries. The paper also evaluates the economic feasibility of

cryptojacking and proposes countermeasures based on its findings.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Cryptojacking Detection** feature because it demonstrates how static JavaScript analysis and machine learning can identify scripts associated with unauthorized cryptocurrency mining. BrowseBlock can adopt similar approaches by extracting JavaScript code-complexity features, examining suspicious script behavior, and using a trained classification model to distinguish potentially malicious mining scripts from benign scripts. The paper's dynamic analysis findings also provide useful indicators for BrowseBlock, including unusually high CPU consumption and suspicious WebSocket communication patterns. These indicators can complement static analysis to improve detection and help BrowseBlock classify websites as safe, suspicious, or potentially malicious. Based on the risk assessment, BrowseBlock can warn users and block known malicious script requests or other supported network activity using browser-extension mechanisms. However, the paper's reported classification results should not be assumed to guarantee the same accuracy in BrowseBlock, particularly because the first experiment used a limited number of distinct cryptojacking scripts. The proposed extension must be evaluated against current scripts, obfuscation techniques, and legitimate resource-intensive JavaScript to reduce false positives and detection gaps.

6 XSS scripting

a) Summary of the Paper: The paper *"Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions"* (2017) by Jinkun Pan and Xiaoguang Mao proposes a hybrid analysis framework to detect DOM-sourced Cross-Site Scripting (XSS) vulnerabilities in browser extensions, particularly Greasemonkey user scripts. The authors explain how attackers can exploit untrusted webpage DOM content to inject malicious JavaScript into vulnerable extension scripts, potentially abusing their privileged capabilities to access cross-origin resources. The proposed framework combines lightweight static analysis with dynamic symbolic execution. Static analysis uses a text filter and an Abstract Syntax Tree (AST) parser to identify suspicious script metadata, overly generic inclusion rules, privileged API usage, potential DOM sources, and dangerous execution sinks such as `eval()`. Scripts that remain suspicious undergo dynamic symbolic execution supported by a Shadow DOM component, which maintains the document structure and

helps generate proof-of-concept exploits to verify vulnerabilities. The authors evaluated their approach on 154,065 Greasemonkey user scripts and identified 58 previously unknown DOM-sourced XSS vulnerabilities. The study demonstrates the effectiveness of combining static and dynamic analysis, while acknowledging that dynamic analysis is computationally expensive and incomplete support for DOM operations may cause missed vulnerabilities.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Cross-Site Scripting (XSS) Detection and Prevention** feature because it demonstrates how static code analysis and DOM-related data-flow analysis can identify potentially dangerous JavaScript execution paths. BrowseBlock can adopt lightweight techniques inspired by the paper, such as examining JavaScript syntax using an AST parser, identifying suspicious DOM inputs, and checking for dangerous execution sinks such as `eval()`. These indicators can be combined with selective DOM monitoring and browser-supported request-blocking mechanisms to assess suspicious activity and prevent access to known malicious scripts where browser capabilities permit. The paper also supports BrowseBlock's approach of filtering potential threats locally before performing more expensive analysis. However, the original framework targets vulnerabilities in Greasemonkey user scripts and relies on dynamic symbolic execution and a Shadow DOM, which are computationally expensive for continuous browser-extension monitoring. BrowseBlock must therefore adapt these techniques into a lightweight, event-driven detection pipeline and evaluate its effectiveness against reflected, stored, and DOM-based XSS attacks. Furthermore, detecting suspicious DOM changes alone cannot guarantee prevention of JavaScript that has already executed; prevention requires appropriate browser-level controls and careful risk-based enforcement.

7 Clickjacking

a) Summary of the Paper: The paper *"Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking"* (2016) by Prof. Dipti Pawade, Abhilasha Lahigude, Divya Reja, and Prof. Era Johri presents ClickProtect, a JavaScript-based Google Chrome extension designed to detect and mitigate clickjacking attacks. The paper focuses on two attack techniques: invisible or transparent iframe overlays and fake cursors that mislead users into clicking unintended elements. The

proposed extension consists of two main components: a revealing unit that scans webpage source code to identify clickable elements, checks their visibility properties, and detects cursor customization; and an extenuation unit that warns users about potential attacks and takes preventive actions. When an invisible iframe or clickable element is detected, ClickProtect increases its visibility and displays a warning message. When a fake cursor is detected, the extension restores the original cursor and notifies the user. The experimental results demonstrate how the extension identifies invisible elements and suspicious cursor modifications, helping users recognize potentially deceptive interactions. The authors also identify the need to extend the solution to additional clickjacking variants and tapjacking attacks.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Clickjacking Detection and Prevention** feature because it demonstrates a browser-extension-based approach to identifying deceptive webpage elements that trick users into performing unintended actions. BrowseBlock can adopt similar techniques by inspecting clickable elements, detecting suspiciously transparent or hidden iframes, examining CSS visibility and stacking properties, and identifying attempts to hide or replace the original cursor. When suspicious behavior is detected, BrowseBlock can display a warning, highlight potentially deceptive elements, and help users avoid unsafe interactions. The paper's two-component architecture also provides a useful design reference for separating threat detection from user notification and preventive actions. However, checking iframe visibility and cursor customization alone cannot reliably detect every clickjacking attack, and legitimate websites may also use transparent elements or custom cursors. BrowseBlock should therefore correlate multiple indicators, minimize false positives, and evaluate its detection methods against different clickjacking techniques. Where possible, browser-supported security controls and website-level protections such as Content Security Policy frame-ancestors directives can complement the extension's detection mechanisms.

8 Browser Extension Abuse

a) Summary of the Paper: The paper *"CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions"* by Xiaoyu Cheng, Zhi Wang, Wanpeng Li, Wei Zeng, and Xin Yang proposes a framework for identifying Chrome browser extensions that request more permissions than they actually

require. The authors develop CPDetector, which extracts declared permissions from the `manifest.json` file and compares them with Chrome extension API usage identified through static JavaScript analysis. The framework also examines host permissions to identify extensions requesting unnecessarily broad access to websites. Using a dataset of 199,058 Chrome Web Store extensions collected as of May 2025, the researchers found that 49.3% (98,072 extensions) exhibited excessive permission authorization. Their analysis also identified 33,998 extensions with excessive host permissions among 129,953 extensions examined for this aspect. To help users understand these risks, the authors developed PermissionGuard, a Chrome extension that identifies over-privileged extensions and displays permission risk warnings. The study highlights how excessive permissions can expose browsing history, cookies, and website data to unauthorized access, emphasizing the importance of the principle of least privilege in browser extension security.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Browser Extension Abuse Detection** feature because it demonstrates how excessive permissions and broad website access can indicate potential security and privacy risks associated with browser extensions. BrowseBlock can adopt similar techniques by analyzing an extension's `manifest.json`, identifying requested API and host permissions, and comparing them with the APIs and website domains referenced in its source code. Permissions such as `cookies`, `history`, and broad host access like `<all_urls>` can be highlighted for further risk assessment when they appear unnecessary for the extension's stated functionality. BrowseBlock can also incorporate permission-risk scoring, extension metadata checks, and user-facing warnings to help users identify potentially over-privileged extensions. However, the paper primarily detects excessive permissions rather than proving that an extension is malicious. BrowseBlock should therefore treat permission mismatches as risk indicators, not definitive evidence of abuse, and combine them with additional evidence such as suspicious script injection, unauthorized data collection, remote code behavior, and changes in extension versions. The paper also notes limitations in static analysis, so dynamic behavior and context-dependent API usage may require additional investigation.

9 Browser Notification Scams

a) Summary of the Paper: The paper *"When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising"* by Karthika Subramani, Xingzi Yuan, Omid Setayeshfar, Phani Vadrevu, Kyu Hyung Lee, and Roberto Perdisci (2020) investigates the misuse of web push notifications to distribute malicious advertisements and scams. The authors introduce **PushAdMiner**, a system designed to collect web push notifications, identify advertising campaigns, and discover potentially malicious campaigns. The system uses an instrumented Chromium browser to monitor notification permissions, Service Worker activity, notification content, network requests, and the URLs users reach after clicking notifications. It then applies text similarity, URL-path comparison, clustering, and threat-intelligence services such as Google Safe Browsing and VirusTotal to identify suspicious campaigns. The researchers collected 21,541 web push notification messages and identified 572 advertising campaigns containing 5,143 advertisements. They reported that 51% of the collected push advertisements were malicious. The study also demonstrated that conventional ad blockers and URL blocklists could miss malicious notification campaigns, highlighting the need for improved detection techniques.

b) Relevance to BrowseBlock: This paper is directly relevant to BrowseBlock's **Browser Notification Scam Detection** feature. It demonstrates how web push notifications can be abused to deliver fake security alerts, fraudulent offers, phishing links, and technical-support scams, even when users are no longer visiting the website that originally requested notification permission. BrowseBlock can apply ideas from the paper by examining notification titles, message bodies, destination URLs, domain information, and redirect chains to identify suspicious notifications. URL reputation checks can be combined with local detection rules and the existing URL phishing detection model to assess the linked destination. BrowseBlock can also identify warning patterns such as fake account-security alerts, misleading rewards, urgent payment warnings, and notifications that redirect users to unrelated domains. When a notification is suspicious, the extension can warn the user and assess its destination before navigation, where browser APIs permit. The paper also highlights the limitations of relying exclusively on URL blocklists, supporting BrowseBlock's approach of combining multiple detection signals. However, PushAdMiner primarily focuses on collecting, measuring, and analyzing malicious advertising campaigns rather than implementing

a lightweight, real-time browser notification protection extension. Therefore, BrowseBlock can use the paper as a research foundation, while developing and evaluating its own real-time detection and prevention mechanism. Browser notification interception and blocking capabilities must also be verified against the specific browser APIs and permissions available to the extension.

10 Credential Harvesting

a) Summary of the Paper: The article *"Web Scraping with Playwright and Python"* by Scrapfly explains how to use Playwright, a browser automation framework, with Python to interact with modern websites and extract information from dynamically rendered web pages. Unlike basic HTTP scraping tools, Playwright can control a real browser engine, execute JavaScript, interact with page elements, and wait for content to appear. It supports browser navigation, element selection through locators, interaction with forms and buttons, and extraction of page content. These capabilities make it suitable for websites that rely on JavaScript to generate or modify their content. Playwright can also be used to automate repeatable browser interactions and test website behavior. Its official Python documentation describes navigation handling, browser contexts, locators, and browser interactions, providing a foundation for building automated web testing and analysis workflows.

b) Relevance to BrowseBlock: This tool is relevant to BrowseBlock's **Credential Harvesting Detection** feature because it can automate the inspection of websites that may imitate legitimate login pages to steal usernames, passwords, and other sensitive information. BrowseBlock can use Playwright in its testing and analysis environment to load suspicious websites, inspect dynamically generated login forms, identify password fields and credential-submission forms, and observe form actions, page URLs, and navigation behavior. Playwright's locator APIs can identify elements using labels, placeholders, roles, and CSS selectors, while its navigation and network-event capabilities can help investigate redirects and requests associated with form submission. These observations can support the development and validation of detection rules that compare the displayed website identity with the actual domain receiving submitted credentials. Combined with BrowseBlock's URL phishing classifier, domain reputation checks, and risk-scoring system, these features can help identify fake login pages and evaluate whether suspicious credential-

harvesting attempts should trigger a warning or blocking action. However, Playwright is best suited to controlled analysis, automated testing, and dataset generation rather than continuous monitoring of every user's browsing session. The production browser extension should perform lightweight, event-driven checks using browser-extension APIs, while Playwright can test those checks against legitimate login pages and simulated phishing pages. Testing must use synthetic credentials and controlled environments; Playwright alone cannot determine whether a login page is malicious or guarantee that credentials will be intercepted.

Literature Review Outcome

SI. No.	Features	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
1	URL / Fake QR Phishing	Google Safe Browsing	XGBoost-Based URL Phishing Detection	Advantages: Does not require heavy GPU power. Reported to be more accurate than classical ML models such as Random Forest. Disadvantages: Multiple datasets are needed for training, validation and testing.	XGBoost model algorithm.
2	HTML / JavaScript Redirects	Google Safe Browsing; Chrome webNavigation and declarativeNetRequest	Zeng et al., "Hidden Path: Understanding the Intermediary in Malicious Redirections" (2022)	Advantages: Monitors and reconstructs redirect chains and identifies HTTP, JavaScript, and HTML redirects, including delayed and multi-step redirects. Disadvantages: Large-scale crawling and analysis can be too resource-intensive for direct browser-extension use. BrowseBlock also requires navigation monitoring, URL reputation checks, risk scoring, and rule-based blocking.	Detects and reconstructs HTTP, JavaScript, HTML, delayed, and multi-step redirect chains. Use Chrome webNavigation for real-time navigation monitoring, URL reputation checks for risk assessment, and declarativeNetRequest for rule-based blocking.

SI. No.	Features	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
3	Cross-Site Scripting (XSS)	NoScript; Content Security Policy (CSP)	Pan & Mao, "Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions"; RXSS Protect (2024)	Advantages: Pan and Mao's dynamic symbolic execution can detect DOM-sourced XSS. RXSS Protect focuses on reflected XSS in URLs. Disadvantages: Dynamic symbolic execution is expensive, while RXSS Protect uses a separate ML server.	Source-to-sink analysis; AST/candidate filtering; reflected-XSS URL and payload analysis. Add lightweight runtime DOM/source-sink monitoring, local XSS heuristics, risk-based warning/blocking, and Playwright testing.
4	Browser Notification Scams	Google Chrome notification protection and Safe Browsing; AdGuard – Block Push API	Subramani et al., "When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising" (2020)	Advantages: Analyzes notification abuse, sender/origin information, notification-related campaigns, and landing URLs. Shows that malicious web-push advertising can bypass traditional ad blockers and URL filters. Disadvantages: PushAdMiner requires instrumented browser infrastructure and is not directly suitable for a normal browser extension.	Analyze notification abuse, sender/origin, campaigns, and landing URLs. Add notification-permission monitoring, suspicious page-behavior analysis, domain/URL reputation checks, selective warning/blocking, and controlled Playwright testing.
5	Browser Extension Abuse	Chrome Web Store / Browser Extension Security	CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions	Advantages: Static permission analysis detects excessive permissions and compares requested permissions with actual API usage. Disadvantages: Mainly focuses on static permission abuse; does not cover suspicious behavior, webpage manipulation, or script injection.	Static permission extraction, permission/API mismatch detection, over-privilege warnings, and runtime security monitoring.

SI. No.	Features	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
6	Cryptojacking	MinerBlock	"Analyzing In-browser Cryptojacking"	Advantages: Detects browser mining using WebSocket and mining-behavior patterns; uses CPU-usage analysis. Disadvantages: Dynamic analysis can be resource-intensive in real time.	Use lightweight CPU-threshold checks and WebAssembly/Web Worker detection. Also support redirect-chain monitoring and reconstruction, identification of HTTP/JavaScript/HTML redirects, and detection of delayed and multi-step redirects.
7	Clickjacking	ClearClick	"Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking"	Advantages: Detects deceptive elements through clickable-element and iframe analysis; examines visibility and cursor properties. Disadvantages: May not detect all framing attacks.	Clickable-element detection, iframe detection, visibility/opacity analysis, cursor-property analysis, overlay and element-position analysis, and transparent-element detection.
8	Email Phishing Detection	Email spam filters / Gmail spam protection	"A Machine Learning Approach for Phishing Email Detection Using Random Forest"	Advantages: Uses feature-based email analysis and Random Forest classification. Disadvantages: Relies on conventional ML-based detection and does not include RAG + LLM, URL-risk fusion, or email-authentication signals used in BrowseBlock.	Use Random Forest for feature-based phishing-email classification. Extract URL count, suspicious URLs, email length, urgency/credential keywords, HTML links, sender-domain mismatch, and URL characteristics to classify emails as phishing or legitimate.

SI. No.	Features	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
9	Malicious File Downloads	VirusTotal; antivirus / endpoint scanners	N. Srndic and P. Laskov, "Hidost: A Static Machine-Learning-Based Detector of Malicious Files," EURASIP Journal on Information Security (2016)	Advantages: Performs static analysis without executing files, extracts structural features for ML classification, and supports format-specific analysis. Disadvantages: Mainly evaluated on PDF and SWF; does not directly cover EXE, DOC/DOCX, or ZIP files; uses Random Forest rather than XGBoost; and does not include browser interception or threat-intelligence checks.	Static file analysis; format-specific structure extraction; feature extraction and selection; feature-vector generation; ML-based classification; download interception and holding; EXE/PDF/DOC/DOCX/ZIP-specific analysis; XGBoost classification; rule-based detection; hash and threat-intelligence checks; and allow/block decisions with logging.
10	Credential Harvesting	Microsoft Defender SmartScreen; Microsoft Edge	Web scraping with Playwright (2026)	Advantages: HTML and JavaScript can be evaluated with Playwright. Disadvantages: Not specified in the source document.	Use Playwright to detect external domains loading scripts and block scam pop-ups.